

Tutorial: File System

Q1. Consider a file system on a disk that has both logical and physical block sizes of 512 bytes. Assume that the information about each file is already in memory. For each of the three allocation strategies (contiguous, linked, and indexed), answer these questions:

a. How is the logical-to-physical address mapping accomplished in this system? (For the indexed allocation, assume that a file is always less than 512 blocks long.)

b. If we are currently at logical block 10 (the last block accessed was block 10) and want to access logical block 4, how many physical blocks must be read from the disk?

Solution:

Logical-to-Physical Mapping (Contiguous, Linked, Indexed)

System assumptions:

- Block size = 512 bytes
- Logical and physical block sizes are the same
- File metadata (starting block, index blocks) is already in memory

Contiguous Allocation

a. Mapping:

Let LA = logical byte address, $B = 512$, and Z = starting physical block of the file. Then:

- $X = LA \div B \rightarrow$ logical block number
- $Y = LA \bmod B \rightarrow$ offset within that block
- Physical block = $Z + X$
- Byte displacement = Y

Example:

For $LA = 2060$ bytes:

- $X = 2060 \div 512 = 4$
- $Y = 2060 \bmod 512 = 2060 - 4 \times 512 = 2060 - 2048 = 12$
- So, physical block = $Z + 4$, offset = 12

b. Accessing logical block 4 (from having accessed block 10):

Contiguous supports direct access $\rightarrow$ you read 1 physical block.

Linked Allocation

a. Mapping:

In linked allocation, each file block contains a pointer to the next. To access logical block X , you must start at the first block and follow X pointers:

- Traverse X blocks, reading each one
- Then within that block, use offset $Y = LA \bmod 512$

b. Accessing logical block 4 (from block 10):

You cannot go backwards. You must restart at block 1 (say 1 is the starting block) and follow pointers:

- Read blocks $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 = 4$ reads

Indexed Allocation (single-level)

a. Mapping:

An index block (already in memory) contains the physical block numbers for each logical block. To access logical block X:

- Lookup $\text{index}[X] \rightarrow$ gives physical block
- $\text{Offset} = Y = \text{LA} \bmod 512$

b. Accessing logical block 4 (from block 10):

Since index is in memory, just read the data block for logical block 4 $\rightarrow$ 1 read

Q2. Consider a file currently consisting of 100 blocks. Assume that the file control block (and the index block, in the case of indexed allocation) is already in memory. Calculate how many disk I/O operations are required for contiguous, linked, and indexed (single-level) allocation strategies, if, for one block, the following conditions hold. In the contiguous-allocation case, assume that there is no room to grow at the beginning but there is room to grow at the end. Also assume that the block information to be added is stored in memory.

- | | |
|--|---|
| a. The block is added at the beginning. | b. The block is added in the middle. |
| c. The block is added at the end. | d. The block is removed from the beginning. |
| e. The block is removed from the middle. | f. The block is removed from the end. |

Solution:

Disk I/O for Block Addition/Removal (Contiguous, Linked, Indexed)

Assumptions:

File has 100 blocks initially

- FCB (and index block, if any) is in memory
- The block to be added is already in memory
- Only necessary block reads/writes and pointer/index updates count

Contiguous Allocation

Operation	Explanation	Disk I/O
Add at beginning	Read 100 blocks, shift forward, write 101 blocks	$100R + 101W = 201$
Add in the middle	Shift ~50 blocks	$50R + 51W = 101$
Add at end	Write the new block	$1W = 1$
Remove from beginning	Shift 99 blocks backward	$99R + 99W = 198$
Remove from middle	Shift ~49 blocks backward	$49R + 49W = 98$
Remove from end	Simply free the last block	0

Linked Allocation

Operation	Explanation	Disk I/O
-----------	-------------	----------

Add at beginning	Write new block, update FCB	$1W = 1$
Add in the middle	Traverse 50 blocks, update 2 pointers	$50R + 2W = 52$
Add at end	Traverse, update last pointer, write new block	$1R + 2W = 3$
Remove from beginning	Update FCB to point to second block	$1R = 1$
Remove from middle	Traverse 51 blocks, update previous block's pointer	$51R + 1W = 52$
Remove from end	Traverse 99 blocks, update pointer of second last	$99R + 1W = 100$

Indexed Allocation (Single-Level, index in memory)

Since the index block is kept in memory, updating it does *not* cost disk I/O—only writing or reading data blocks counts.

Operation	Explanation	Disk I/O
Add at beginning	Shift index entries in memory + write data block	$1W = 1$
Add in the middle	Shift index + write data block	$1W = 1$
Add at end	Update index + write block	$1W = 1$
Remove from beginning	Shift index entries in memory	0
Remove from middle	Shift index entries	0
Remove from end	Remove last entry in index	0

Operation	Contiguous I/O	Linked I/O	Indexed I/O
Add at beginning	201 ($100R + 101W$)	1 ($1W$)	1 ($1W$)
Add in the middle	101 ($50R + 51W$)	52 ($50R + 2W$)	1 ($1W$)
Add at end	1 ($1W$)	3 ($1R + 2W$)	1 ($1W$)
Remove from beginning	198 ($99R + 99W$)	1 ($1R$)	0
Remove from middle	98 ($49R + 49W$)	52 ($51R + 1W$)	0
Remove from end	0	100 ($99R + 1W$)	0

Q3. Consider a file management system that uses indexed allocation scheme on a hard disk with a 256B per blocks storage capacity. The blocks can be classified either as index blocks or data blocks. Every file is assigned a primary index block similar to the inode structure in Unix file system. The data block does not contain any header/ control information and the entire 256B can be used for data storage. Each primary index block contains a 32B header entry and 28 block index entries each of which is 8B. Each such block index entry carries the address of another block which can be either another index block or a data block. Out of the 28 block index entries, first 25 of them are pointing to data blocks directly and the remaining three are single indirect, double indirect and triple indirect index blocks like the one used in Unix inode. The 28 block index entries of a single indirect block are all pointing to data blocks, where as the 28 block index entries of a double indirect block are all pointing to addresses of another set of single indirect index block. Similarly the triple indirect index block can also be defined with 3 levels of indirection. Block mapping for a file is done by fully completing each lower level (entry 1-25) before going to next level of indirection (entry 26,27 and 28). Indexed Allocation with Multi-Level Indirection (Unix Inode)

- a. What is the maximum size of a file that can be mapped only with direct 25 data blocks of the primary index block (single, double and triple indirect entry is null)?
- b. A file was assigned a valid entry in the single indirect entry of its primary index block and 20 out of 28 entries of that single indirect index block were valid. What can be the maximum size of the file?
- c. What is max file size possible?

Setup (same as before):

- Block size = 256 bytes
- Primary index block: 32B header + 28 entries (8B each)
- Entries 1–25: direct pointers to data blocks
- Entry 26: single indirect
- Entry 27: double indirect
- Entry 28: triple indirect
- Indirection mapping is hierarchical: fill all direct first, then single, then double, then triple

(a) Max file size with only direct pointers (no indirects)

- Direct entries = 25
- Each data block = 256 bytes
- Max size = $25 \times 256 = 6,400$ bytes

(b) Single indirect used with 20 valid pointers (plus all direct)

- 25 direct = $25 \times 256 = 6,400$ bytes
- 20 valid in single indirect = $20 \times 256 = 5,120$ bytes
- Total = $6,400 + 5,120 = 11,520$ bytes

(c) Maximum possible file size using all levels

Let ($n = 28$) (number of pointers in each index block). Then:

- Direct part: (25×256)

- Single indirect: $(n \times 256)$
- Double indirect: $(n^2 \times 256)$
- Triple indirect: $(n^3 \times 256)$

Compute for $(n = 28)$:

- Direct = 6,400 B
- Single = $(28 \times 256 = 7,168)$ B
- Double = $(28^2 \times 256 = 784 \times 256 = 200,704)$ B
- Triple = $(28^3 \times 256 = 21,952 \times 256 = 5,619,712)$ B

Total max size = $6,400 + 7,168 + 200,704 + 5,619,712 = 5,833,984$ bytes ≈ 5.56 MB

Note: For practice, refer to the exercise questions in *Operating System Concepts* by **Abraham Silberschatz, Peter Baer Galvin, and Greg Gagne.**